



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure, Environment, Actuators, Sensors.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width`, `self.height`, `self.agent_pos`, `self.walls`, `self.food_positions`, `self.opponents`, `self.score`, `self.steps`, and `self.collison`.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This is a Multi-Agent environment. The key clue is `self.opponents`, which initializes a list of adversarial agents that can move independently of the main agent. Even if a run sets `num_opponents=0`, the baseline environment is designed to support

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all percepts an agent has received over time, not just the current percept.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

`get_percept()` represents the Sensors part of PEAS, because it is the mechanism that returns what the agent can perceive from the environment.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This environment is Partially Observable. The returned dictionary gives useful information like `agent_pos`, `opponent_positions`, `smells_food`, `hit_wall`, `collision`, `score`, and `remaining_food`, but it does not expose the full hidden state of the world, such

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The component being updated is the Performance measure, specifically `self.score`. Hitting a wall changes the outcome score, so it affects how well the agent is performing.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is important because PEAS performance should measure task success in the external world, not how hard the agent thinks internally. If the metric rewarded or penalized internal processing effort, the evaluation would become about

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hiding `self.toxic_traps` from the agent's sensors makes the environment more strongly partially observable. The trap still exists in the world state, but the agent cannot directly perceive it, so it must act under uncertainty rather than having full

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

smells_toxin improves rationality because it gives the agent extra evidence about nearby danger before committing to an action. That helps the agent choose the action with the highest expected utility, since it can now trade off food collection

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic vacuum-world exploit was the agent gaming a flawed performance measure by looping on an easy local behavior, such as repeatedly sucking or staying in one place, instead of actually cleaning the whole environment. The point of that

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING